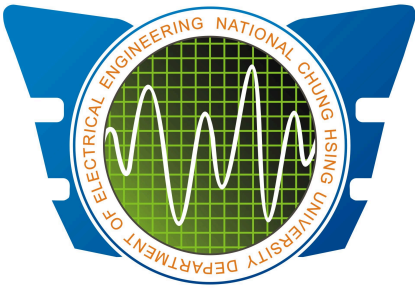
國立中興大學 電機工程學系

專題報告

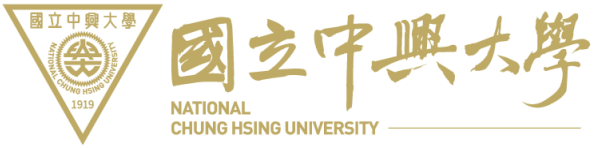


自製 RV32IMZicsr 處理器：從 RISCOF 合規與 FPGA 部署到循序管線中推測性協同處理器卸載之安全邊界形式化刻畫

A RISC-V RV32IMZicsr Processor with Multi-Layer Verification: From RISCOF Compliance and FPGA Deployment to Formal Characterization of the Safety Boundary for Speculative Coprocessor Offloading in In-Order Pipelines

電機四 伍柏霖

指導老師：范志鵬 教授



摘要

本專題設計並實作了一顆自訂的 RV32IM + Zicsr 處理器：先以官方的 RISCOF 合規測試（62/62 通過）與 FPGA 實機部署驗證其正確性，再在其上擴充自訂向量處理器與動態分支預測器兩項微架構，平行化版本達約 2.84× speedup、分支預測器使執行時間降低 39.7%。在完成這些工作的過程中，一個具體的時序問題逐漸浮現：當多週期協同處理器指令與分支預測驅動的推測執行同時存在時，什麼樣的時序關係決定了「推測性污染」在結構上是否可能發生、以及何時必須引入顯式的 kill/commit 復原機制才能維持正確性？此問題在業界真實的協同處理器擴充介面規格（CV-X-IF）中已被要求，但其主流參考實作（CVA6）公開文件明確將此路徑標示為未支援，顯示這是一個具體、有實務依據的實作缺口。

針對這個問題，本專題採用「具體安全層 → 具體時序層 → 抽象邊界層」的三層互補形式化方法：先以 k-induction 證明目前設計在鏡像模型上的結構性安全（Theorem A1/A2），再以不干預電路功能的 Ghost Timing Model 技術從真實 RTL 時序關係中量出關鍵的推測距離參數，最後將此具體數值代入參數化的抽象安全模型，以有界模型檢查（BMC）求解安全邊界的位置。在所評估的參數空間中，形式化掃描結果支持三個邊界關係：$F \leq D + 1$（基準情境）、$F \leq D + 1 + L_{LAT}$（納入協同處理器延遲後）、以及 $F \leq D + 1 + S$（納入管線 stall 後）；三者在邊界內全數 PASS，邊界外第一個參數點均可產生形式化反例。真實 baseline CPU 在 stall-free 條件下的具體推測距離為 $D = 0$，經 Ghost Timing Model 直接由 RTL 時序量測得出，座落於安全區內側。

本報告依序記錄這條從實作到研究的完整路徑：處理器與驗證平台的建立、微架構探索的量化成果、以及最終從中開展的形式化安全邊界分析。報告也如實說明目前的限制：上述邊界關係為有限離散參數組合下、經形式化掃描支持的猜想，並非符號化的數學歸納證明；kill/commit 復原機制僅於抽象模型層級評估，尚未落地為真實硬體實作；Theorem A1/A2 之證明建立於與原始 RTL 忠實對應（人工假設、尚未形式化驗證）的鏡像模型之上。完整限制討論見第五章。

目錄

第一章　動機與背景 4

第二章　系統設計與驗證方法 5

第三章　微架構探索 10

第四章　推測性協同處理器卸載之安全邊界形式化分析 14

第五章　討論與限制 23

第六章　結語 25

參考文獻 26

附錄 A　RISCOF 測試結果彙總 29

附錄 B　Theorem A1/A2 性質描述 29

附錄 C　Ghost Timing Model 設計說明 30

附錄 D　參數掃描完整矩陣 30

附錄 E　代表性反例 31

附錄 F　FPGA 互動式展示系統細節 31

第一章　動機與背景

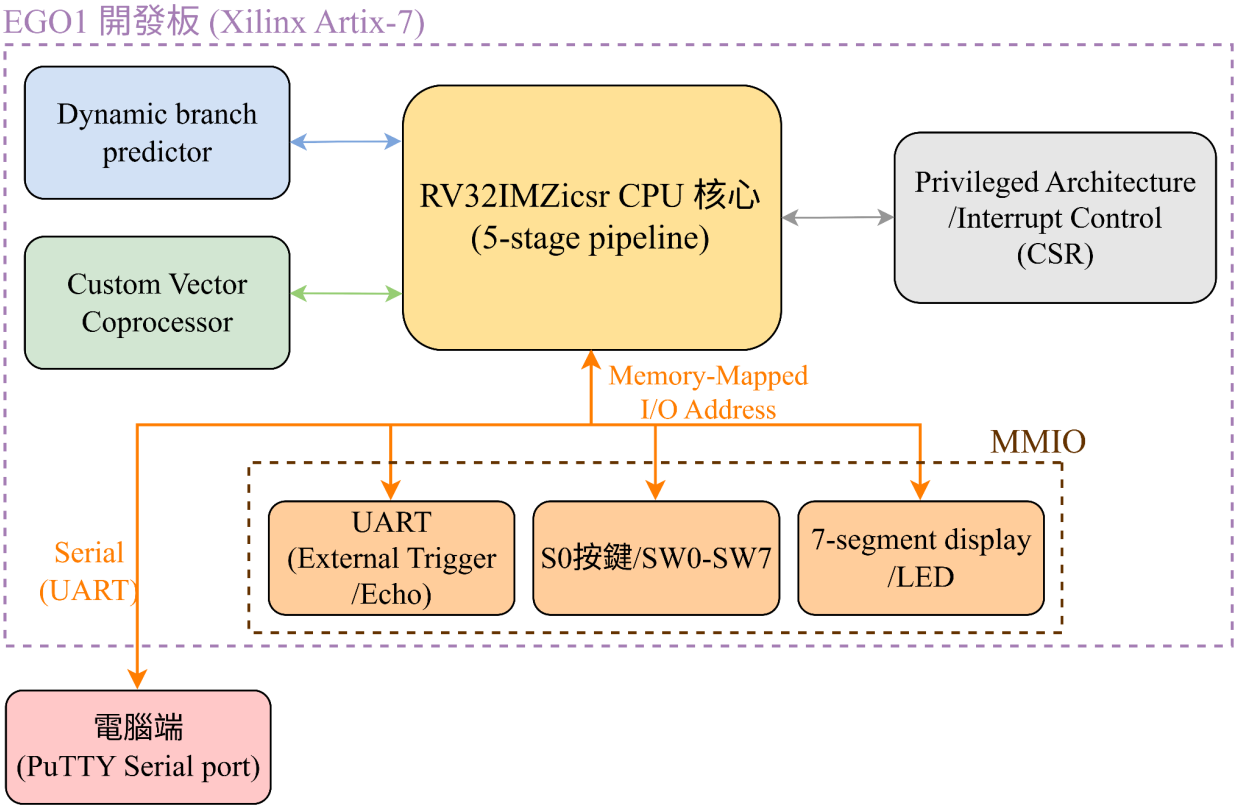
處理器設計的教學專題，多半止步於「做出一顆能跑的 CPU」，較少涉及業界真正使用的驗證方法論（與 Spike 參考模型進行合規比對、硬體實機部署、形式化驗證，Formal Verification），也較少對自己做出來的微架構設計進行量化的效能／資源權衡分析。本專題希望走完一條更完整的路徑：先做出一顆自訂的 RV32IM + Zicsr 處理器 [1]，用 RISC-V 官方的 RISCOF 框架 [2] 跟參考模型（spike）逐位元比對確保 ISA 語意正確，再實際部署至 FPGA 面對真實硬體的時序與外部訊號問題；在核心穩定後，進一步擴充兩項代表性的微架構——自訂向量處理器與動態分支預測器——並對其效能與資源成本進行量化分析。

在完成這些工作、把驗證方法論延伸至形式化方法的過程中，一個具體的問題逐漸浮現：處理器同時具備了分支預測與自訂的多週期協同處理器（向量單元）之後，當分支預測造成錯誤路徑（wrong-path）指令提前進入管線，而協同處理器本身又有多週期執行延遲時，錯誤路徑上的操作是否可能在 flush 生效前就已經啟動、進而造成推測性污染？這個問題並非憑空提出——業界真實的協同處理器擴充介面規格 CV-X-IF [3] 明文規定 commit_kill 訊號語意，要求被 kill 的指令不得改變 architectural state，以保護推測執行下的正確性；但查證發現其主流參考實作 CVA6 [4] 公開文件目前明確將 offloaded-instruction killing 標示為 unsupported feature，公開實作描述中 commit_kill 不會被設置——這代表 CV-X-IF 所規範的保護機制，存在一個具體、有公開文件依據的實作缺口，也是本專題後續研究工作的直接動機來源。

本報告接下來依實際工作順序展開：第二章說明作為研究平台的處理器設計與驗證方法；第三章記錄兩項微架構探索的量化成果；第四章是本專題的核心研究部分，完整呈現從上述問題出發、最終建立的安全邊界形式化分析；第五章解讀結果與討論本專題的限制；第六章總結全文。

第二章　系統設計與驗證方法

本章說明作為後續微架構探索與核心研究的具體平台——一顆功能完整、經 RISCOF 官方合規測試與 FPGA 實機部署雙重驗證的 RV32IM + Zicsr 處理器，其系統的整體架構如圖一。本章的目標是證明這個研究平台是真實、完整、且經過驗證的，為第三、四章的研究工作建立可信的基礎。



圖一、系統整體架構

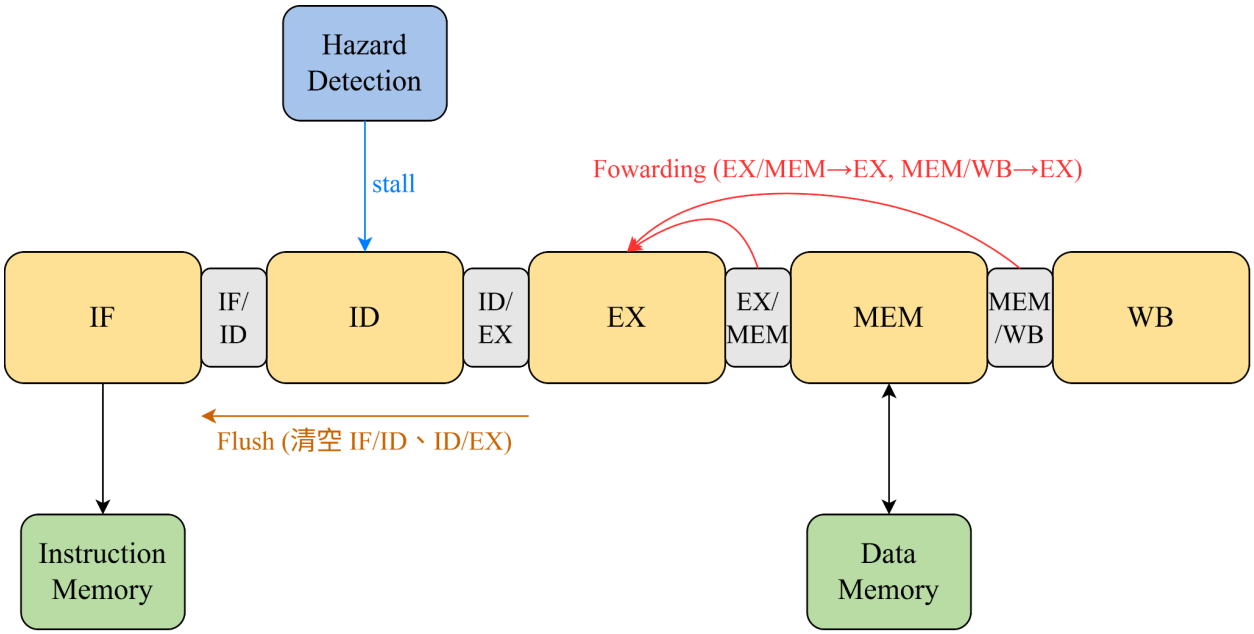
2.1 處理器核心架構

處理器實作 RV32IM + Zicsr（含機器模式特權架構，privileged architecture），採用經典五級管線（pipeline）（IF/ID/EX/MEM/WB），具備完整的資料轉發（forwarding）與冒險偵測（hazard detection）機制，其架構如圖二。

其中特權架構支援三種獨立的中斷/例外來源：

- **同步例外**：ecall、ebreak、指令/資料位址未對齊
- **軟體中斷**：透過記憶體映射位址觸發，並串接真實的外部 UART 訊號作為觸發來源
- **計時器中斷**：實作標準的 mtime/mtimecmp（64-bit）比較機制

中斷優先權遵循 RISC-V 規格（軟體中斷 > 計時器中斷），圖三為 trap_cause 的片段程式碼，明確反映優先權鏈（由高到低：中斷 > 指令未對齊 > ecall/ebreak > load/store 未對齊）。



圖二、處理器核心架構

```
reg [31:0] trap_cause_ex;
always @(*) begin
    if (msi_pending)            trap_cause_ex = {1'b1, 31'd3}; // machine software interrupt (帶中斷位元)
    else if (mti_pending)       trap_cause_ex = {1'b1, 31'd7}; // machine timer interrupt (帶中斷位元)
    else if (instr_misalign_ex) trap_cause_ex = 32'd0;   // instruction address misaligned
    else if (is_ecall_ex_r)     trap_cause_ex = 32'd11;  // environment call
    else if (is_ebreak_ex_r)    trap_cause_ex = 32'd3;   // breakpoint
    else if (load_misalign_ex)  trap_cause_ex = 32'd4;   // load address misaligned
    else if (store_misalign_ex) trap_cause_ex = 32'd6;   // store address misaligned
    else                        trap_cause_ex = 32'd0;
end
```

圖三、特權架構優先權鏈

2.2 驗證方法論總覽

RISC-V 是一套開放的指令集架構規格 [1]，其合規性驗證由官方的 RISCOF 框架 [2], [5] 提供標準化方法，透過與參考模型（spike）逐位元比對 signature 輸出，確保實作的 ISA 語意正確性。本研究在此基礎上，從 ISA 行為、實際硬體與形式性質三個面向建立互補的驗證流程，如表一所示。

表一、三層驗證方法論總覽

層次	方法	目的
模擬驗證	RISCOF + spike 參考模型比對；自行設計的定向測試	確保 ISA 語意正確
實機驗證	FPGA（EGO1, Xilinx Artix-7）部署，含獨立硬體自動檢查器	確保在真實時序、真實 I/O 條件下行為正確
形式化驗證	Yosys + SymbiYosys（SAT/SMT-based），k-induction 與有界模型檢查	對以 k-induction 成功證明的關鍵不變量提供無界的形式保證；對參數化 timing model 的邊界猜想，則採用有界形式化掃描（BMC），而非窮舉式的無界證明

這三層驗證具有互補關係——模擬驗證找出的多數真實 bug（詳見 2.3 節），後來也被形式化驗證用來設計對應的性質，反向確認問題的根因與修法的完備性。在硬體驗證領域，形式化方法提供比模擬測試更強的保證：k-induction 可對關鍵不變量提供無界的數學證明，而有界模型檢查（BMC）[6] 則對參數化的時序模型提供有界但系統性的驗證覆蓋。本研究採用的開源工具鏈（Yosys + SymbiYosys）與應用於 RISC-V 核心的既有實踐一脈相承，例如 Wolf 提出的 riscv-formal 框架 [7] 及其在工業界的應用案例 [8]，雖然本研究採用自行設計的 SVA 性質、而非 riscv-formal 的 RVFI 介面。

此外，本研究在驗證流程中大量採用「刻意設計的健全性檢查」（植入已知錯誤，確認驗證流程真的能偵測到違反），此作法與形式化驗證文獻中突變測試（mutation testing）量化驗證品質的方法論精神一致，例如 Funck 等人針對 RISC-V 形式化驗證框架提出的系統性突變分類方法 [9]，本研究的健全性檢查可視為此類方法論在小規模、手動情境下的應用。

形式化驗證結果彙整於表二。

表二、形式化驗證成果彙總

性質	驗證等級	結果
計時器比較邏輯正確性	k-induction	通過
記憶體讀寫一致性	k-induction	通過
管線暫存器 reset 完整性	k-induction	通過
中斷優先權邏輯	組合邏輯完整窮舉	通過
分支預測器安全性質（predict_target 資訊洩漏，[10], [11]）	BMC	發現缺陷並修補後通過

Theorem A1/A2 是與 baseline RTL 邏輯對應之 flush-safety 鏡像模型（mirror model）上的 k-induction 證明結果；鏡像模型與原始 RTL 的忠實對應為一項明確列出的 assumption，詳見第四章與附錄 B。

四次健全性檢查中，兩次意外發現了具教學價值的方法論問題（斷言誤用實作本身作為規格描述、性質涵蓋範圍不足），並在修正後正確地展現偵測能力，確認整套驗證流程並非形式上通過、而是真正發揮了檢驗作用。

2.3 FPGA 部署

將處理器部署至 EGO1 開發板（Xilinx Artix-7, XC7A35T），除了完成基本的燒錄流程，另外設計了一套獨立硬體自動驗證檢查器：CPU 從 BRAM 讀取一組預先寫入的測試資料、執行運算、寫回結果，檢查器再獨立比對結果是否正確，並透過板載 LED 回報 PASS/FAIL，不需要透過 UART 或除錯器，實現不依賴外部工具的自我驗證。此外，實作了真實的 UART 外部觸發機制：電腦端透過序列埠主動送出資料，FPGA 硬體直接觸發軟體中斷（不經過 CPU 自我觸發），CPU 被中斷、讀取資料、並將結果回傳，達成真正的雙向即時互動。

成功部署後，硬體自動檢查器完整運作（16/16 測試模式通過，LED 正確回報）；UART 外部觸發雙向互動經實測穩定運作；計時器中斷經模擬與實機雙重驗證。部署過程中定位並修正的真實硬體問題包含：Block RAM 推斷失敗、reset 極

性誤判（兩次）、時序不足、Verilog 隱性宣告在 Vivado 與 Verilator 間的行為落差、7 段顯示器映射錯誤、以及一個因非同步訊號跨時脈域未同步而導致的資料競爭問題（採用標準的兩級正反器同步器設計 [12] 解決）。

2.4 互動式展示系統

為了讓 FPGA 實機成果不只停留在「十六進位燈號需要口頭翻譯」的展示形式，另外設計了一套觀眾可直接操作、並能即時獲得文字回饋的互動系統：新增 S0 按鍵與 SW0–SW7 撥動開關作為實體輸入（皆經防彈跳、同步器處理），CPU 可輪詢讀取；並設計 send_string、send_hex 兩個通用副程式，讓 CPU 在展示流程的 12 個階段轉換時，主動透過 UART 送出對應的文字說明，與 LED 燈號標記一致，搭配 7 段顯示器即時顯示輸入數值。系統已完成實機測試，完整的硬體介面設計細節（防彈跳計數器、sticky flag 機制）與 12 階段流程對照表見附錄 F。

此系統的建置過程中，也發現並修正了三個具技術參考價值的問題：資料需在執行期間動態寫入資料記憶體、而非編譯期間內嵌於指令記憶體（Harvard 架構限制）；巢狀副程式呼叫若未保護返回位址暫存器，會導致控制流程錯亂；以及兩個模組（分支預測器、向量單元）中共計 17 個訊號的 reset 賦值遺漏，僅在重新合成至 Vivado 時才被揭露。

第三章　微架構探索

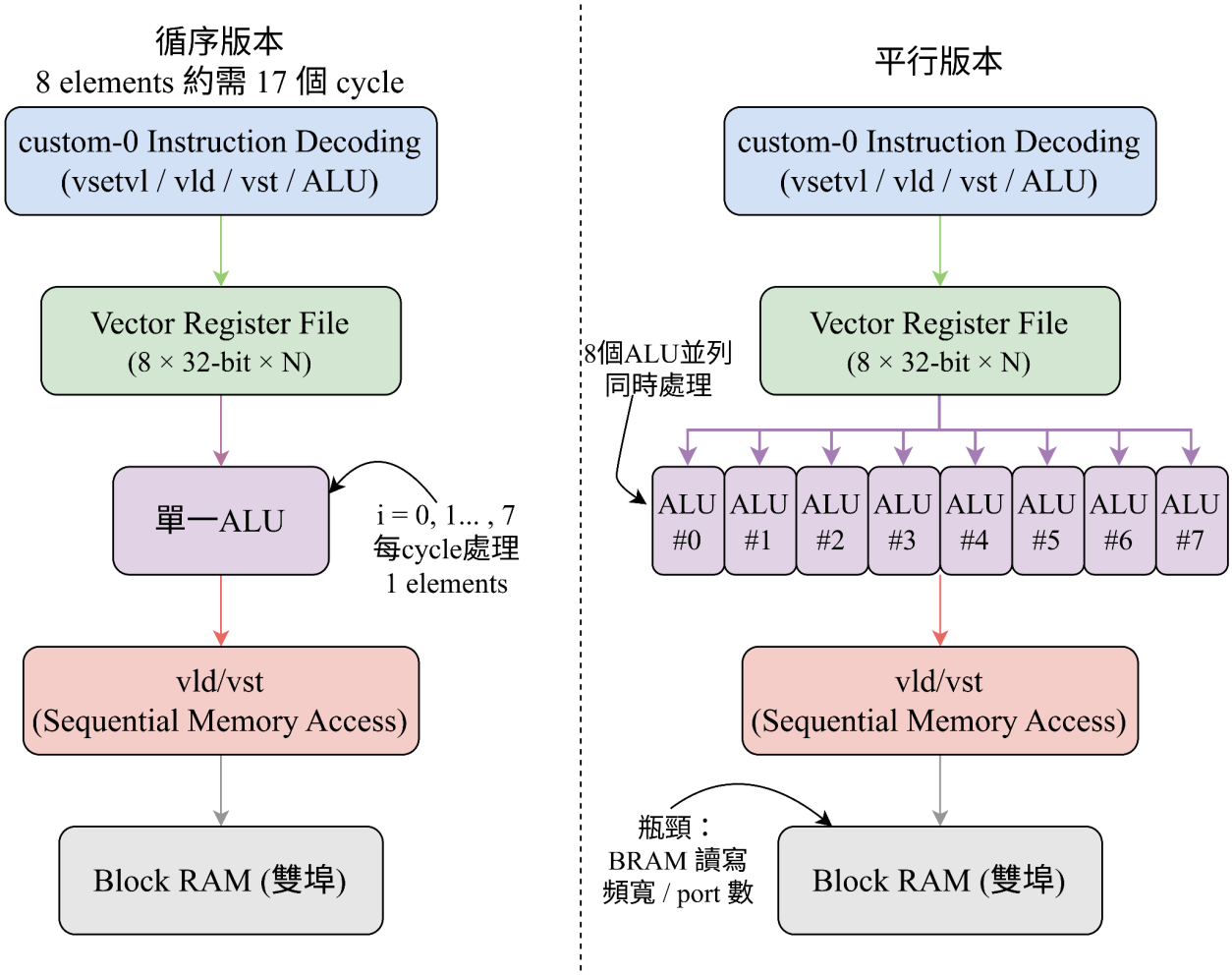
在第二章確立的驗證平台上，本章對兩項具代表性的微架構擴充——自訂向量處理器與動態分支預測器進行量化探索，並附帶一項輔助性的形式化安全案例研究。本章的角色是為第四章的核心研究提供實際微架構背景，兩項擴充本身並非與安全邊界研究並列的獨立研究主題。

3.1 自訂向量處理器

設計了一組透過 RISC-V custom-0 opcode 實作的輕量向量指令集（vsetvl、vld、vst、六種向量-向量/向量-純量 ALU 運算，共 9 條指令），採用獨立協同處理器架構，透過管線凍結機制（而非清空）與主管線整合。這種「借用 custom-N opcode 空間掛載自訂協同處理器」的設計模式，與業界既有的 RoCC（Rocket Custom Coprocessor）介面規格 [13] 採用相同的基本思路；Berkeley 自行研發的 RISC-V 向量協同處理器 Hwacha [14] 正是透過同一介面掛載的實際案例，可作為本研究向量單元設計的既有參考案例，惟本研究的介面設計為自行實作、規模也遠較 Hwacha 精簡。為了對其微架構決策進行量化研究，實作了兩種微架構版本：

- **循序版本**：向量運算逐元素處理，8 個元素需要約 17 個 cycle
- **平行版本**：向量-向量/向量-純量運算改為單一 cycle 完成（8 組獨立硬體展開），記憶體存取（vld/vst）維持循序（受限於真實 Block RAM 僅雙埠的硬體限制）

圖四為兩種版本的架構對比圖，表三呈現量化後的比較成果：



圖四、循序版本 vs 平行版本架構對比

表三、向量處理器循序版本與平行版本量化比較

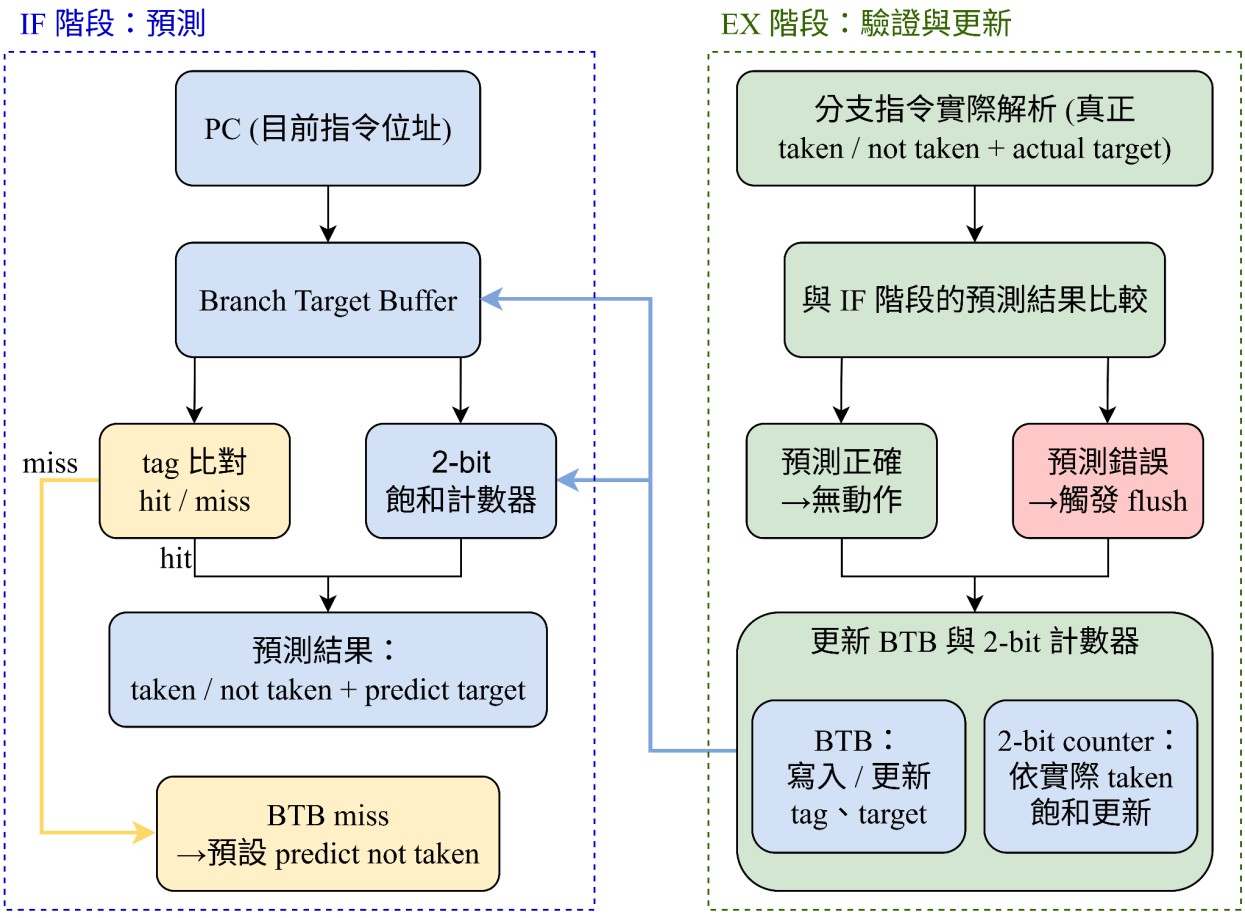
項目	循序版本	平行版本
執行 cycle 數（400 個向量指令）	~6,952	~2,452（-64.7%）
Slice LUTs	15,175	17,216（+13.4%）
DSP 使用量	15	36（+140%）
時序（WNS）	5.215 ns	5.424 ns

結果顯示效能提升未達理論上限：以 17-cycle 的循序運算部分作為理想化 baseline，純運算平行化的理論上限約為 17 倍 speedup；實際整體執行 cycle 數降低

64.7%，相當於約 2.84 倍 speedup（6952/2452 ≈ 2.84×），而非理想化的 17 倍，原因在於記憶體存取（vld）仍為循序瓶頸，隨運算部分加速後反而佔據更高比例的執行時間——此為 Amdahl 定律 [15] 在客製化硬體加速器設計上的具體案例，同時 DSP 資源用量的大幅增加（+140%）具體量化了平行化的硬體成本。Worst Negative Slack 由 5.215 ns 變為 5.424 ns，兩種版本均保有正的 timing margin，顯示在目前 constraint 下平行版本未造成 timing closure 惡化。

3.2 動態分支預測器

實作 2-bit 飽和計數器 [16] + 分支目標緩衝區（BTB）[17] 的動態分支預測器，並加入位址標籤（tag）比對機制，避免位址別名碰撞造成誤判。預測發生於 IF 階段，驗證與更新發生於 EX 階段，猜對時完全不需要清空管線，只有猜錯才觸發既有的 flush 機制。圖五為其架構圖。



圖五、動態分支預測器架構

3.2.1 量化成果

在一組用於評估分支預測效果的基準測試（1000 次迴圈、單一高機率分支）下，執行時間由 50,155 ns 降至 30,215 ns，改善 39.7%，每次迴圈精確節省 2 個 cycle（對應消除的 flush 懲罰），數據與理論預期完全吻合。

3.3 效能與資源權衡總結

上述兩項微架構擴充的量化結果，共同呈現了一個一致的工程主題：微架構層級的加速決策，其實際效益往往受限於未被同步優化的次要瓶頸（向量單元的記憶體存取）、或伴隨可量化的硬體成本（DSP 用量暴增）。這個主題也為第四章的安全邊界研究提供了直接的微架構背景——動態分支預測器正是後續問題中「分支預測驅動的推測執行」的具體來源，而向量處理器則是「多週期協同處理器」的具體實例。

3.4 輔助案例研究：分支預測器安全性分析

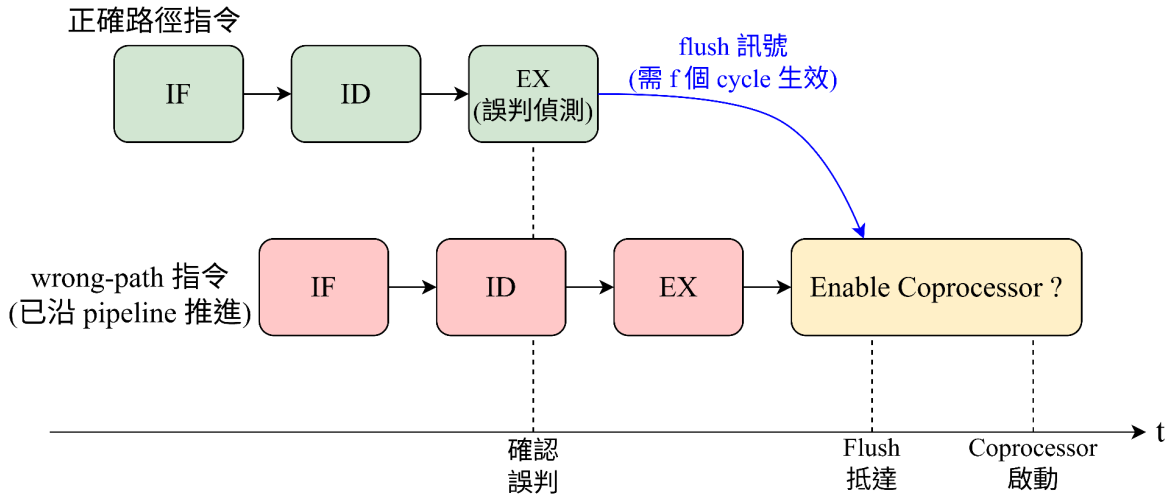
作為驗證方法論的一個輔助案例研究（supporting case study），另外將形式化驗證延伸至安全性質分析：針對分支預測器的位址別名機制進行威脅建模，發現並修補了一個真實存在、此前未被察覺的資訊洩漏設計缺口（predict_target 訊號在 tag miss 時未受保護）。此類分支目標緩衝區資訊洩漏問題，與 Spectre 系列攻擊 [10] 及其在 RISC-V 平台上的具體重現 [11] 所描述的威脅模型性質相同。此案例與本研究第四章的核心研究問題性質不同、各自獨立，故在此明確標註為輔助發現。

第四章　推測性協同處理器卸載之安全邊界形式化分析

4.1 問題起源

第三章的處理器已經同時具備了分支預測與自訂的多週期協同處理器（向量單元）。當這兩者進一步結合時，一個自然浮現的時序問題是：若分支預測造成錯誤路徑（wrong-path）指令提前進入管線，而協同處理器本身又有多週期執行延遲，錯誤路徑上的操作是否可能在 flush 生效前就已經啟動、進而造成推測性污染？錯誤路徑指令與 flush 訊號之時序問題示意如圖六。

這個問題並非憑空提出。業界真實的協同處理器擴充介面規格 CV-X-IF [3] 明文規定 commit_kill 訊號語意，要求被 kill 的指令不得改變 architectural state，即是 kill 訊號一旦生效，協同處理器必須捨棄該指令已計算的中間結果，不將其寫回任何架構可見的狀態，以保護推測執行下的正確性。但查證發現其主流參考實作 CVA6 [4] 公開文件目前明確將 offloaded-instruction killing 標示為 unsupported feature，公開實作描述中 commit_kill 不會被設置——這代表 CV-X-IF 所規範的保護機制，存在一個具體的實作缺口。本章接下來要分析的，正是分支解決距離、flush 延遲、協同處理器延遲、與管線 stall 這幾個時序因素之間的關係：什麼樣的條件會讓推測性污染在結構上變得可能，又是什麼條件能確保它不會發生？



圖六、分支誤判後之時序問題示意圖

4.2 相關研究定位

處理器層級的形式化驗證涵蓋多週期功能單元與分支預測，並非全新的研究領域。這條技術脈絡可追溯至 Burch 與 Dill 提出的管線控制邏輯 flushing 驗證技術 [18]，Velev 與 Bryant 發表於 DAC 的經典工作 [19] 即是將此技術延伸至同時處理多週期功能單元、例外、與分支預測的超純量處理器驗證，證明了管線實作與 ISA 層級規格之間的功能等價性（functional equivalence）。本研究與此類既有工作的差異在於問題的性質：既有工作關注「管線實作是否正確實現 ISA 語意」，而本研究關注的是一個更細緻的時序刻畫問題——在什麼樣的具體時序關係下，推測性卸載機制需要顯式的 kill/commit 協定才能維持安全，以及這個需求邊界能否被參數化、精確地求解。

在協同處理器介面規格方面，CV-X-IF 明確規定了 kill/commit 語意，但屬於協定文字規範而非形式化證明；RoCC 介面 [13] 提供了既有的自訂加速器掛載模式，第三章提到的 Hwacha [14] 即是透過同一介面掛載的實際案例。在時序相關的形式化工作方面，MINOTAuR [20] 針對包含推測執行的 RISC-V 核心進行了時序可預測性（timing predictability）的形式化分析，關注點與本研究的安全邊界不同；CHERIoT-Ibex [21] 則展示了針對 microarchitectural 層級的 observational correctness 與 liveness 形式化驗證方法。

就查證所及的公開文獻範圍，尚未見到針對「推測性協同處理器卸載的參數化時序安全邊界」這個特定問題的形式化刻畫。表四整理了上述工作與本研究在四個不同面向上的定位比較。

表四、相關研究定位比較

工作	多週期功能單元	分支推測	形式化方法	參數化時序安全邊界
Velev & Bryant [19]	✓	✓	✓（functional equivalence）	未涉及
MINOTAuR [20]	未涉及	✓	✓（timing predictability）	未涉及（關注時序可預測性，非安全邊界）

工作	多週期功能單元	分支推測	形式化方法	參數化時序安全邊界
CV-X-IF 規格 [3]	✓（協定層級）	✓（kill/commit 語意）	協定文字規範，非形式化證明	未涉及
CHERIoT-Ibex [21]	部分（microarchitectural）	—	✓（observational correctness + liveness）	未涉及
本研究	✓	✓	✓	✓（見本章）

註：表中「未涉及」代表就本研究查證所及的摘要與內容判斷該篇工作未以此為目標，非對原文的逐句查核；完整結論仍需以原始文獻為準。

本研究採用「具體安全層（Concrete Safety Layer）→ 具體時序層（Concrete Timing Layer）→ 抽象邊界層（Abstract Boundary Layer）」的三層互補架構：先針對與 baseline RTL 對應的鏡像模型，用形式化方法證明目前設計所關注的安全性質，再用不干預電路功能的 Ghost Timing Model 技術，從真實 RTL 的時序關係中量出關鍵的推測距離參數，最後將這個具體數值代入參數化的抽象安全模型，求解安全邊界的位置。以下各節依這個順序展開。

4.3 具體安全層：Theorem A1 與 A2

Theorem A1（Flush Completeness）證明：一旦 flush 訊號觸發，下一個 cycle，pc_out_o 與 is_vec_op_o 恆為 0。Theorem A2 將此性質延伸至完整的 vec_start 判斷式（涵蓋全部向量指令類型），證明 is_vec_op_o 作為單一「主閘」（master gate）的角色，使 A1 幾乎自動蘊含 A2。兩者皆以 k-induction 完成證明，並搭配刻意植入錯誤（例如移除 is_vec_op_o 清除邏輯）的健全性檢查，確認驗證流程確實能偵測到違反。

> 關於鏡像模型的說明：因特定版本 Yosys 對「非同步 reset 條件與同步訊號混合判斷式」的相容性限制，本研究最終在改寫過判斷式寫法的模組化鏡像模型上完成 k-induction 證明，並假設該鏡像模型在 A1/A2 所涉及的訊號行為上與原始 RTL 一致；此對應關係尚未經形式化驗證，明確列為一項需要讀者知悉的 assumption。完整性質描述見附錄 B。

4.4 參數定義

在說明後續觀察前，先正式定義使用的四個核心時序參數，如表五。

表五、核心時序參數定義

符號	定義
D	分支解決（branch resolution）到假想協同處理器啟動點（hypothetical coprocessor launch）之間的有效管線推進距離
F	誤判偵測（misprediction detection）到 flush 生效之間的延遲，以 cycle 計
L_{LAT}	抽象模型中，協同處理器自身的運算完成／結果可見延遲（completion/visibility latency）
S	使管線推進被延後的有效 stall cycle 數

直觀上，D 表示危險指令還需要走多久才會抵達協同處理器；F 則表示 flush 還需要多久才真正生效。當 flush 生效時刻與危險指令抵達啟動點時刻落在同一 cycle 時，本研究定義 flush 事件優先於 launch 事件生效，這也是後續邊界關係中「等號」被歸類為安全（PASS）的依據。其中 D 是本研究方法論的核心量測對象——透過 4.5 節的 Ghost Timing Model 技術，直接從真實 RTL 的訊號時序關係中量出，而非人工假設或代入的參數；下文的 D 均指 stall-free 條件下的結構性管線推進距離，納入 stall 後的效應另立第 4.8 節分析，不併入 D 本身的定義。

4.5 具體時序層：Ghost Timing Model

4.6–4.8 節的抽象邊界模型將推測距離 D 當作參數處理，但 D 在真實 RTL 上對應的具體數值，此前從未被正式量測過，只是人工假設的常數。Ghost Timing Model 技術透過引入一個純粹用於觀測、不影響原始電路行為的「ghost」訊號——其精神與形式化驗證文獻中「ghost variable / ghost code」技巧一致 [22]——鏡射 id_ex_register 的閂鎖行為，但刻意不套用 flush 清除，藉此觀測「若沒有保護機制，指令原本會落在哪裡」，而不干預真實 CPU 的行為。

研究過程中發現：flush 與 stall 同時發生時，ghost 訊號的更新會被凍結，代表 D 並非固定常數，而是 $D = f$(管線時序, $stall$ 歷史)的函數 。在明確限定

stall-free 條件下，Ghost Timing Model 以有界 BMC 形式化證明：flush 觸發時刻閂鎖的 is_vec_op_in 數值，會精確反映在 ghost 訊號的下一個 cycle——由此正式導出 $D = 0$（stall-free 條件）。這並非「假設 $D = 0$」，而是「從具體 RTL 時序關係中形式化導出 $D = 0$」。

4.6 觀察一：基準情境下的安全邊界

以參數化抽象模型（僅含 D、F 兩個變數，不含 kill 機制）進行系統性形式化掃描，涵蓋 $D \in \{0, 1, \ldots, 8\}$ 、 $F \in \{0, 1, \ldots, 10\}$ 共 99 組獨立驗證。結果精確吻合以下關係：

$$Safe(D, F) \Leftrightarrow F \leq D + 1$$

99 組驗證中，邊界內（$F \leq D + 1$）全數 PASS，邊界外的第一個參數點即產生形式化反例（FAIL）。真實 baseline CPU（$D = 0$，第 4.5 節導出）代入後得 $F \leq 1$，即 flush 訊號必須在誤判偵測後 1 個 cycle 內生效才能維持安全——這與 Theorem A1 的敘述（flush 觸發下一拍即完成清除，對應 $F = 0$）一致。本研究的 cycle indexing 將 flush 觸發當拍記為 $F = 0$；F 每增加 1，代表 flush 生效延後一個 clock cycle，安全邊界中的「+1」則反映管線狀態需要到下一個 clock edge 才真正完成更新。

4.7 觀察二：納入協同處理器延遲後的邊界

與 4.6 節以「wrong-path 指令是否啟動」作為安全判準不同，本節納入 kill/commit 機制後，安全性定義為：被 kill 的協同處理器指令，在其運算結果變得架構可見（architectural state）之前即被攔截，不留下可觀測的影響。

將抽象模型擴充為包含 kill/commit 機制與協同處理器自身執行延遲 L_{LAT} 兩項因素，涵蓋 $D \in \{0, 1, \ldots, 8\}$、$F \in \{0, 1, \ldots, 10\}$ 、 $L_{LAT} \in \{0, 2, 4\}$ 共 297 組獨立驗證，分別驗證了純粹的 safety 性質，以及 safety+liveness 的組合性質（後者需額外處理閂鎖暫存器導致的狀態空間複雜度，改採 BMC 而非 k-induction）。結果如下：

$Safe(D, F, L_{LAT}) \Leftrightarrow F \leq D + 1 + L_{LAT}$，且 safety 與 liveness 的安全區完全重合

safety 與 liveness 安全區完全重合這件事本身具有工程意義：這代表在本研究評估的模型中，kill/commit 機制只要能維持正確性（safety），就不會犧牲進度保證（liveness）——兩者並非通常預期的權衡關係。一個反直覺但可解釋的現象是：協同處理器自身延遲越大（L_{LAT} 越大），安全邊界反而越寬鬆——因為較慢的協同處理器讓 kill 訊號有更多時間趕上，在真正造成危害之前完成攔截。此現象僅適用於本節所定義、具有 kill/commit 攔截語意的抽象模型，不代表一般實際協同處理器延遲越大就必然越安全。

4.8 觀察三：納入管線 Stall 後的邊界

將管線 stall 因素 S 加入抽象模型（此模型不含 kill 機制，S 是相對於 4.6 節基準模型的獨立延伸，從未與 4.7 節的 L_{LAT} 在同一個模型中被聯合驗證），涵蓋 $D \in \{0, 1, \ldots, 8\}$、$F \in \{0, 1, \ldots, 10\}$、$S \in \{0, 1, 2, 3\}$、共 384 組獨立驗證。結果如下：

$$Safe(D, F, S) \Leftrightarrow F \leq D + 1 + S$$

驗證過程本身也留下一個具一般性的方法論教訓：第一版模型採用 stall_duration $\leq S$ 的假設方式，結果顯示 S 完全不影響邊界位置。推演後發現，因為 stall 只會讓指令抵達變慢、不會提早，stall_duration=0（無 stall）永遠是最壞情況，導致求解器可以永遠選擇這個情況、讓 S 參數失去意義。修正為 stall_duration = S（恰好 S 拍）後，才能真正比較「有無 stall」的差異，得出上述關係。

4.9 形式化策略的迭代精煉

表六呈現本研究在建立上述分析架構的過程中，經歷四次方法論上的迭代。

表六、形式化策略的四次迭代

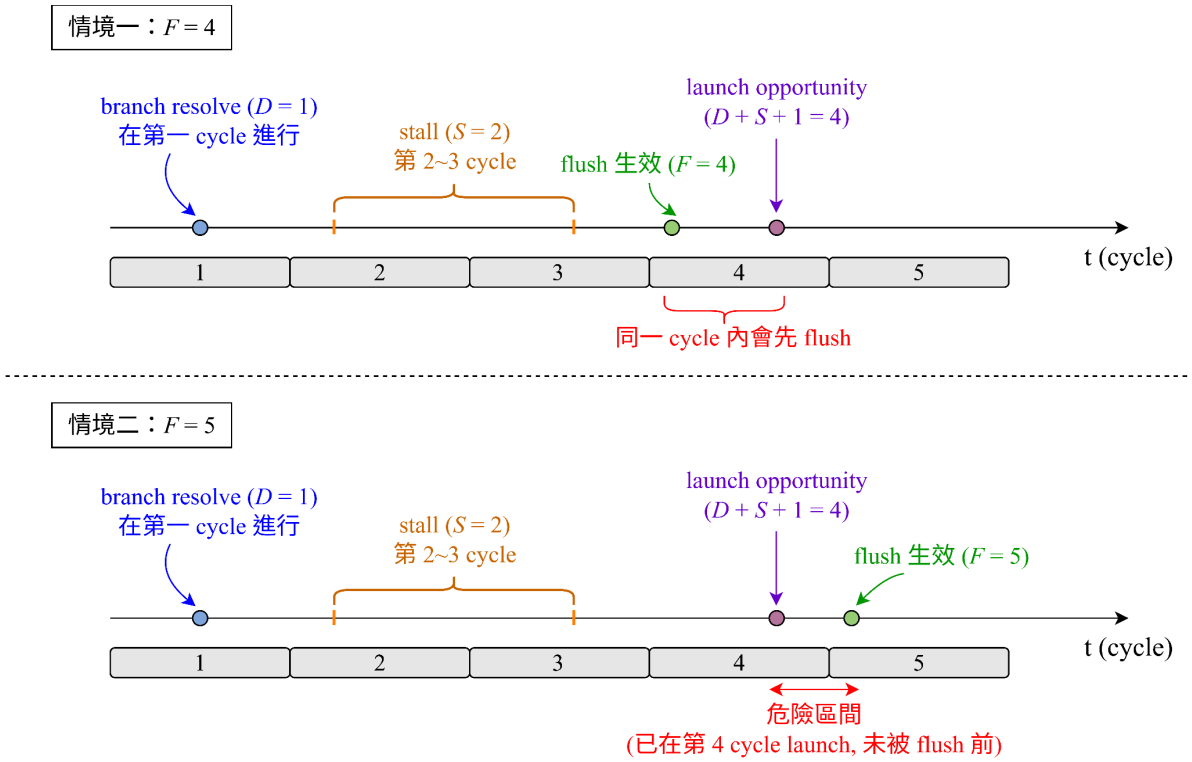
迭代	發現的問題	修正方式
事件範圍界定（Event Scoping）	初始模型隱含假設單一生命週期事件，未明確界定分析範圍	明確限定 4.6–4.8 節分析範圍為單一、孤立的推測事件
性質語意（Property Semantics）	「不可再有 vec_start」的性質誤將合法的後續指令（flush 後從正確路徑重新擷取）也一併禁止，導致假陽性	改為明確辨識「特定的 wrong-path 指令」，而非全面禁止協同處理器活動

迭代	發現的問題	修正方式
觀測／保護耦合（Observation /Protection Coupling）	觀測用的訊號恰好也被保護機制本身定義，導致性質結構性恆真（vacuity）[23]	採用分離觀測與保護的 Ghost Timing Model，確保觀測機制不與保護機制耦合
Stall 依賴性（Stall Dependence）	發現 flush 與 stall 同時發生時，D 並非固定常數	限定 stall-free 條件下求解具體 $D = 0$，stall 對 D 的影響另立第 4.8 節分析

4.10 代表性反例走查

為了讓 4.6–4.8 節的邊界公式不只停留在符號層面，本節以一組實際驗證過的具體參數（$D = 1$, $S = 2$）完整走過安全與違反兩種情境，說明邊界公式背後真正的時序因果關係。

依 4.8 節的關係（$F \leq D + 1 + S$），此參數組合的邊界為 $F \leq 4$。危險指令若未被阻止，會在 $t = 4$ 這個時刻抵達協同處理器啟動點；圖七呈現 $F = 4$（邊界值）與 $F = 5$（超出邊界一格）兩種情境的時序關係。



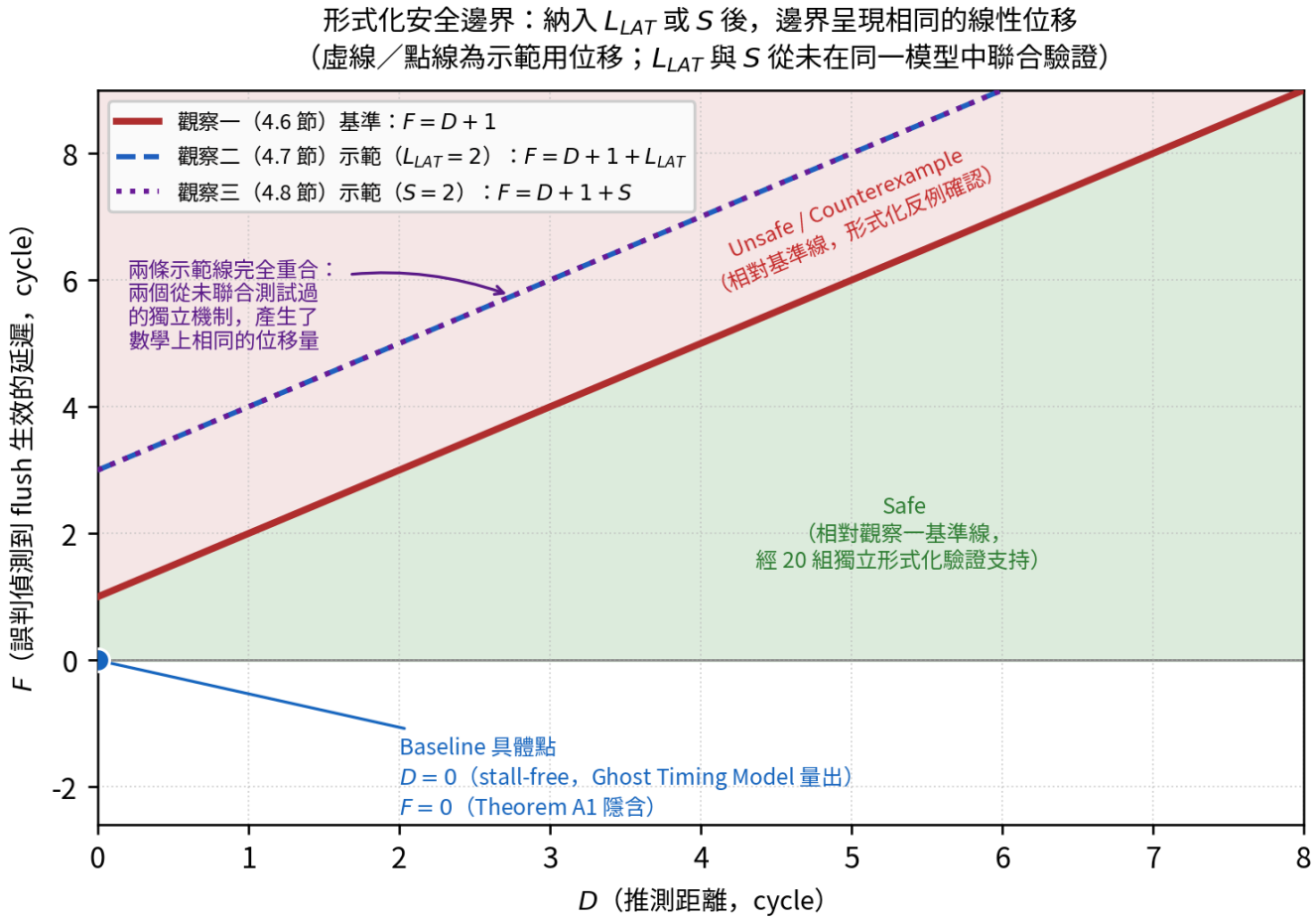
圖七、具體反例走查時序圖（$D = 1$, $S = 2$）

情境一（$F = 4$，PASS）：flush 生效與危險指令抵達啟動點發生於同一 cycle；依 4.4 節定義的事件優先順序，同一 cycle 內 flush 先於 launch 生效，保護機制準時完成，指令從未真正在無保護狀態下啟動。

情境二（$F = 5$，FAIL）：flush 生效的時刻晚於危險指令抵達啟動點的時刻，兩者之間出現一段「指令已經啟動、flush 卻還沒生效」的危險區間——這正是形式化驗證回報反例的具體物理意義：不是求解器隨意判定，而是真實存在一段結構上會發生推測性污染的時間區間。

4.11 統一討論

圖八以視覺化方式統一呈現 4.6-4.8 節的觀察結果。



圖八、形式化安全邊界示意圖

圖中以 4.6 節求出的基準線 $F = D + 1$（紅線）為核心，另外疊加兩條示範用位移線：分別對應 4.7 節（取 $L_{LAT} = 2$ 為例）與 4.8 節（取 $S = 2$ 為例）。兩條示範線在圖中完全重合，正是本圖想要凸顯的發現。藍點標示本研究真實 baseline CPU 的具體座落位置（$D = 0$, $F = 0$，第 4.3、4.5 節），落在安全區內側。

4.7 節與 4.8 節的兩條邊界延伸——分別納入協同處理器延遲 L_{LAT} 與管線 stall S 後，兩者在數學上呈現完全相同的線性位移量，儘管兩者從未在同一個形式化模型中被聯合驗證過（4.7 節模型不含 S，4.8 節模型不含 L_{LAT}）。此結果支持一個較一般性的時序解釋：

> **凡能延後潛在錯誤路徑操作抵達協同處理器的因素，可能都可被視為有效推測距離的增加，並以相同的線性方式擴大安全邊界。**

但此推論仍需在其他延遲機制（例如更複雜的 forwarding/hazard 交互作用，或 L_{LAT} 與 S 同時存在的聯合情境）與具體 RTL 上進一步驗證，方能視為真正獨立於實作細節的一般性結論——這個保留態度也正是第五章要完整討論的限制之一。

第五章　討論與限制

本報告採取如實呈現限制的立場。以下限制在後續延伸或口頭報告中應主動說明。

5.1 方法論限制

1. **邊界公式為經驗證支持的猜想，非符號化證明**：$F \leq D + 1 + S$ 等公式，是對有限離散參數組合的窮舉驗證結果，並非對任意非負整數的數學歸納證明。

2. **Stall 對推測距離的影響尚未在真實 RTL 上驗證**：4.8 節的 $F \leq D + 1 + S$ 是在抽象模型層級求出的關係；尚未反過來在真實 RTL 上，用 Ghost Timing Model 驗證帶 stall 情況下 $D + S$ 的具體數值。

3. **Kill/commit 協定尚未落地為真實硬體實作**：第 4.7 節的結論建立在抽象參數化模型上，vector_unit.v 目前並未真正具備接受 kill 訊號的機制。

4. **僅涵蓋單一獨立推測事件**：連續、時間上相鄰或重疊的多次分支誤判之間的交互影響，需要 epoch/tag 機制才能正確處理，目前明確排除在研究範圍外。

5. **部分性質驗證於模組化抽出的鏡像模型，而非原始檔案**：因特定版本 Yosys 對「非同步 reset 條件與同步訊號混合判斷式」的相容性限制，Theorem A1/A2 最終在改寫過判斷式寫法的鏡像模組上完成證明，鏡像與原始 RTL 的忠實對應是人工建立、尚未形式化驗證的 assumption。

6. **新穎性主張基於有限的文獻查證**：本研究方向的定位參考了公開的協同處理器擴充規格與相關學術文獻，但未經過系統性、涵蓋完整會議資料庫的正式文獻回顧，故僅能以「就查證範圍所及未見完整處理」表述，而非「絕對首創」。

5.2 結果解讀與工程意義

第四章的分析可以收斂為三個結論。

1. 就本研究所定義的 baseline wrong-path 協同處理器卸載安全性質而言，現有設計在 stall-free 條件下位於安全區內。baseline CPU 在 stall-free 條件下的具體推測距離為 $D = 0$（第 4.5 節由 Ghost Timing Model 導出），代入邊界關係後得 $F \leq 1$ 即可維持安全，而 Theorem A1 保證 flush 觸發下一拍即完成清除（對應 $F = 0$）——目前的設計座落於安全區內側，且這並非巧合，而是經形式化驗證確認的結果。

2. 安全不是無條件成立，而是存在明確的參數條件。第 4.6–4.8 節求解出的三個邊界關係（$F \leq D + 1$ 、$F \leq D + 1 + L_{LAT}$ 、$F \leq D + 1 + S$）都是條件式的：只要 F 超出邊界，危險指令就會在 flush 生效前搶先啟動，第 4.10 節的具體反例已完整示範這個情況。第 4.11 節更進一步發現，協同處理器延遲與管線 stall 這兩個獨立因素，在數學上呈現完全相同的線性位移——代表這個邊界條件具有一致、可預期的結構，而非個別現象的巧合疊加。

3. kill/commit 是跨越安全邊界後的復原機制，目前尚未落地為真實硬體。第 4.7 節在抽象模型層級證明了納入 kill/commit 語意後，安全邊界確實可以延伸，代表如果系統的時序關係跨越了基準邊界，這個機制可以提供有效的補救；但它本身尚未在 vector_unit.v 上實作，也就還沒有實機驗證可以引用。值得一提的是：以本研究 baseline CPU 目前的具體參數來看，即使沒有 kill/commit，系統仍座落於安全區內——這代表 kill/commit 對這顆處理器目前而言並非燃眉之急，但對於任何 D 、F 關係更接近邊界、或協同處理器延遲更長的未來擴充，這個機制的必要性會隨之提高。

這三點合起來，正是本研究希望留下的核心訊息：安全邊界不是抽象的理論產物，而是一條可以套用在具體系統參數上、給出明確答案的工程判斷標準。

第六章　結語

本專題從一顆通過 RISCOF 官方合規測試（62/62）與 FPGA 實機部署雙重驗證的 RV32IMZicsr 處理器出發，逐步延伸至自訂向量處理器與動態分支預測器兩項微架構，分別達成約 2.84 倍的 speedup 與 39.7% 的執行時間改善。最終收斂至一個具體、與既有形式化驗證工作（如 Velev & Bryant [19]）及業界規格（CV-X-IF [3]）明確定位其差異的形式化驗證問題：推測性協同處理器卸載在循序管線中的安全邊界。

在此基礎上，進一步以 k-induction 證明了目前設計的結構性安全（Theorem A1/A2）之後，透過不干預電路功能的 Ghost Timing Model 技術，將抽象邊界模型的參數與真實 RTL 時序建立具體對應（$D = 0$，stall-free 條件）；求解出三個經系統性形式化掃描支持的邊界關係：$F \leq D + 1$ 、$F \leq D + 1 + L_{LAT}$ 、$F \leq D + 1 + S$，並發現協同處理器延遲與管線 stall 這兩個從未被聯合驗證過的獨立因素，在數學上呈現完全相同的線性位移。

目前做到的地方，以及還沒做到的地方，都已在 5.1 節如實列出，這些限制即是下一步工作的起點。後續可以繼續推進的方向如下：

1. 把 stall 對 D 的影響延伸回真實 RTL 驗證
2. 將邊界猜想提升為符號化證明
3. 在 vector_unit.v 上實作 kill 機制並做實機驗證
4. 引入 epoch/tag 機制，擴充至多個相鄰或重疊的推測事件
5. 對鏡像模型與原始 RTL 的對應關係進行正式形式化驗證
6. 探索本方法的推廣性，套用到其他協同處理器介面、甚至檢視業界實作。

例如 CVA6 若未來補上 kill/commit 機制，能否直接套用本研究的邊界公式驗證其設計。這也是本研究希望留下的：一套可以重複使用、檢驗其他系統的分析方法。過程中的多次除錯與方法論修正，為本專題重要的歷程，如實記錄而非隱去。

參考文獻

[1] RISC-V International, "The RISC-V Instruction Set Manual" (RV32I, M extension, Zicsr, and Privileged Architecture specifications).

[2] RISC-V International, "RISCOF: RISC-V Compatibility Framework," GitHub repository.

[3] OpenHW Group, "Core-V eXtension Interface (CV-X-IF) Specification."

[4] OpenHW Group, "CV-X-IF Interface and Coprocessor," CVA6 User Manual, https://docs.openhwgroup.org/projects/cva6-user-manual/, accessed Sep. 2026.

[5] M. A. Elshiemy et al., "Comprehensive Compliance Framework for RISC-V Architectures using the RISCOF Test Platform," IEEE, 2025.

[6] A. Biere, A. Cimatti, E. Clarke, and Y. Zhu, "Symbolic Model Checking without BDDs," in Tools and Algorithms for the Construction and Analysis of Systems (TACAS), LNCS vol. 1579, 1999, pp. 193–207.

[7] C. Wolf, "RISC-V Formal Verification Framework," https://github.com/YosysHQ/riscv-formal, 2016.

[8] L. Bordonaro, E. Rapuano, S. Bocchio, and L. Fanucci, "Integration of the Open-Source RISC-V Formal Verification Framework into the STMicroelectronics Toolchain and Its Application to the STRiVe2.4 CPU," in Applications in Electronics Pervading Industry, Environment and Society (ApplePies 2024), LNEE vol. 1369, Springer, 2025.

[9] M. Funck, S. Ahmadi-Pour, V. Herdt, and R. Drechsler, "Identification of ISA-Level Mutation-Classes for Qualification of RISC-V Formal Verification," in 2023 Forum on Specification & Design Languages (FDL), 2023, pp. 1–8.

[10] P. Kocher et al., "Spectre Attacks: Exploiting Speculative Execution," in 2019 IEEE Symp. Security and Privacy (S&P), 2019, pp. 1–19.

[11] R. Bălucea and P. Irofti, "Software Mitigation of RISC-V Spectre Attacks," in Innovative Security Solutions for Information Technology and Communications (ISCIS 2022), LNCS, Springer, 2024 (arXiv:2206.04507).

[12] S. Beer, R. Ginosar, M. Priel, R. Dobkin, and A. Kolodny, "Metastability and Synchronizers: A Tutorial," IEEE Design & Test of Computers, vol. 28, no. 5, pp. 23–35, 2011.

[13] K. Asanović et al., "The Rocket Chip Generator," Technical Report UCB/EECS-2016-17, EECS Department, University of California, Berkeley, Apr. 2016.

[14] Y. Lee, A. Ou, C. Schmidt, S. Karandikar, H. Mao, and K. Asanović, "The Hwacha Microarchitecture Manual, Version 3.8.1," Technical Report UCB/EECS-2015-263, EECS Department, University of California, Berkeley, Dec. 2015.

[15] G. M. Amdahl, "Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities," in Proc. AFIPS Spring Joint Computer Conf., 1967, pp. 483–485.

[16] J. E. Smith, "A Study of Branch Prediction Strategies," in Proc. 8th Int. Symp. Computer Architecture (ISCA), 1981, pp. 135–148.

[17] J. K. F. Lee and A. J. Smith, "Branch Prediction Strategies and Branch Target Buffer Design," IEEE Computer, vol. 17, no. 1, pp. 6–22, Jan. 1984.

[18] J. R. Burch and D. L. Dill, "Automatic Verification of Pipelined Microprocessor Control," in Proc. Computer Aided Verification (CAV), LNCS vol. 818, 1994, pp. 68–80.

[19] M. N. Velev and R. E. Bryant, "Formal Verification of Superscalar Microprocessors with Multicycle Functional Units, Exceptions, and Branch Prediction," in Proc. 37th Design Automation Conf. (DAC), 2000, pp. 112–117.

[20] A. Gruin, T. Carle, C. Rochange, H. Cassé, and P. Sainrat, "MINOTAuR: A Timing Predictable RISC-V Core Featuring Speculative Execution," IEEE Trans. Comput., vol. 72, no. 1, pp. 183–195, 2023.

[21] L.-E. Ploix, A. Armstrong, T. Melham, R. Lin, H. Wang, and A. Courtney, "Comprehensive Formal Verification of Observational Correctness for the CHERIoT-Ibex Processor," arXiv:2502.04738, Feb. 2025.

[22] J.-C. Filliâtre, L. Gondelman, and A. Paskevich, "The Spirit of Ghost Code," Formal Methods in System Design, vol. 48, pp. 152–174, 2016.

[23] I. Beer, S. Ben-David, C. Eisner, and Y. Rodeh, "Efficient Detection of Vacuity in ACTL Formulas," in Proc. Computer Aided Verification (CAV), 1997.

附錄 A　RISCOF 測試結果彙總

表 A-1　RISCOF 合規測試結果

測試類別	項目數	通過數	通過率
RV32I（基礎整數指令集）	38	38	100%
M 擴充（乘除法）	8	8	100%
Zicsr 與特權架構	16	16	100%
總計	62	62	100%

全部測試皆與 spike 官方參考模型進行 signature 逐位元比對，比對方式為 Verilator 模擬輸出與 spike 輸出經 RISCOF 框架自動判定一致性。

附錄 B　Theorem A1/A2 性質描述

以下以結構化描述呈現 Theorem A1/A2 的性質定義，完整 RTL 原始碼請參照專題原始檔案 caseA_formal.v。

- **Theorem A1（Flush Completeness）**：對任意 cycle t，若 flush 於 t 拍為真，則於 t+1 拍，pc_out_o == 0 且 is_vec_op_o == 0 恆成立。
- **Theorem A2（Wrong-path vec_start 防止）**：對任意 cycle t，若 flush 於 t 拍為真，則於 t+1 拍，完整的 vec_start 判斷式（涵蓋全部 9 種向量指令類型）恆為假。
- **驗證等級**：k-induction，並搭配植入已知錯誤（移除 is_vec_op_o 清除邏輯）的健全性檢查，確認驗證流程能正確偵測違反。
- **已知限制**：因 Yosys 對「非同步 reset 與同步 flush 訊號混合於同一 if 判斷式」的相容性限制，證明實際執行於改寫過判斷式寫法的鏡像模組，而非原

始 id_ex_register.v 檔案本身；鏡像模組與原始檔案在 A1/A2 所涉及訊號行為上的對應關係，屬於尚未形式化驗證的 assumption。

附錄 C　Ghost Timing Model 設計說明

完整 RTL 原始碼請參照專題原始檔案 ghost_timing_model.v。設計要點如下：

- 鏡射 id_ex_register 對 is_vec_op_in 的閂鎖行為，但刻意不套用 flush 清除邏輯，藉此觀測「若無保護機制，指令原本會落在哪裡」。
- 觀測訊號與保護機制（真實的 flush 清除邏輯）完全分離，避免兩者結構性耦合導致性質恆真（vacuity）。
- 驗證限定於 assume(!stall) 的 stall-free 條件，形式化證明 flush 觸發時刻閂鎖的 is_vec_op_in 數值會精確反映在 ghost 訊號的下一個 cycle，導出具體 $D = 0$。
- 驗證等級：有界 BMC（因狀態空間涉及閂鎖暫存器歷史，k-induction 在此模型上呈現 UNKNOWN，改採深度足以涵蓋檢查點的 BMC）。

附錄 D　邊界交界點驗證矩陣

以下表格彙整第 4.8 節各（D, S）組合所對應的安全邊界值，以及邊界交界點的驗證結果，涵蓋 $D = 0$ 至 8 共九條獨立的 D 值線，共 36 個邊界交界點；完整（D, F, S）三維驗證資料另存於專題原始驗證紀錄中。

表 D-1　Stall-Aware 邊界驗證矩陣（D×S 組合下的安全上限 F）

D	$S = 0$	$S = 1$	$S = 2$	$S = 3$
0	$F \leq 1$	$F \leq 2$	$F \leq 3$	$F \leq 4$
1	$F \leq 2$	$F \leq 3$	$F \leq 4$	$F \leq 5$
2	$F \leq 3$	$F \leq 4$	$F \leq 5$	$F \leq 6$

D	$S=0$	$S=1$	$S=2$	$S=3$
3	$F \le 4$	$F \le 5$	$F \le 6$	$F \le 7$
4	$F \le 5$	$F \le 6$	$F \le 7$	$F \le 8$
5	$F \le 6$	$F \le 7$	$F \le 8$	$F \le 9$
6	$F \le 7$	$F \le 8$	$F \le 9$	$F \le 10$
7	$F \le 8$	$F \le 9$	$F \le 10$	$F \le 11$
8	$F \le 9$	$F \le 10$	$F \le 11$	$F \le 12$

每一格數值皆為獨立形式化驗證結果：F 等於表列數值時 PASS，F 等於表列數值加一時 FAIL（形式化反例），36 組交界點全數精確吻合 $F = D + 1 + S$ 的預測，無一例外。

附錄 E　代表性反例

除第 4.10 節完整走查的 $D = 1, S = 2$ 情境外，以下列出另外兩組具代表性的實際驗證邊界交界點，用以對照公式：（1）$D = 0, S = 0$：$F \le 1$ 時 PASS，$F = 2$ 時 FAIL，對應最基本的 stall-free、無延遲情境；（2）$D = 8, S = 0$：$F \le 9$ 時 PASS，$F = 10$ 時 FAIL，對應本研究掃描範圍中 D 值最大的情境，用以確認邊界公式在延伸至較大參數時依然成立、並非僅在小數值下偶然吻合。

附錄 F　FPGA 互動式展示系統細節

完整 12 階段展示流程如表 F-1 所示，每一階段的 LED 燈號（十六進位）與 UART 主動回報的文字說明皆一一對應。

表 F-1　互動式展示系統階段流程

階段	**LED 標記**	**內容**
1	0x01	系統開機，CSR 初始化開始
2	0x02	CSR 初始化完成，準備進入自動檢查

階段	LED 標記	內容
3–4	0x03–0x04	16 項自動檢查執行中／完成（D1 LED 顯示 16/16 通過）
5	0x05	向量單元運算完成，7 段顯示器顯示 checksum
6	0x06	等待撥動開關輸入第一個數值，按 S0 確認
7	0x07	等待撥動開關輸入第二個數值，按 S0 確認
8	0x08	顯示計算結果（兩數值相加）
9–10	0x09–0x0A	計時器中斷武裝、等待觸發
11	0x0B	計時器中斷已觸發，處理完成
12	0x0C	進入 UART 互動待機，可反覆測試雙向回音